

Chapter 44 — String Algorithms

0. What You Will Learn in This Chapter

String Algorithms are different from String DP.

String DP is about states such as:

```
dp[i][j]
dp[l][r]
```

String Algorithms are about direct pattern-based processing of strings using:

- hashing
- frequency arrays
- two pointers
- sliding window
- prefix/suffix logic
- string matching
- lexicographic ordering
- stack
- trie preview
- KMP
- Z algorithm preview
- rolling hash preview
- parsing-like scans
- string construction

This chapter connects many earlier patterns with string-specific problems.

By the end of this chapter, you should understand:

- How to process strings efficiently
- Frequency counting in strings
- Anagram patterns
- Subsequence checking
- Prefix/suffix string tricks
- KMP string matching
- LPS array construction
- Z algorithm concept
- Lexicographic string problems
- Stack-based string construction
- String compression
- String rotation
- Repeated substring patterns
- How to choose between sliding window, hashing, KMP, trie, DP, and rolling hash

Core problems covered in this chapter:

1. Valid Anagram
2. Group Anagrams
3. Find All Anagrams in a String
4. Longest Common Prefix
5. Is Subsequence
6. Backspace String Compare
7. Valid Palindrome
8. Valid Palindrome II
9. Implement strStr

10. Repeated Substring Pattern
 11. KMP Prefix Function
 12. Z Algorithm Preview
 13. Longest Happy Prefix
 14. Rotate String
 15. String Compression
 16. Remove All Adjacent Duplicates
 17. Remove Duplicate Letters
 18. Smallest Subsequence of Distinct Characters
 19. Minimum Remove to Make Valid Parentheses
 20. Custom Sort String
 21. Multiply Strings
 22. Add Strings
 23. Compare Version Numbers
 24. Count and Say
 25. Integer to Roman / Roman to Integer
-

1. What Are String Algorithms?

String algorithms are techniques for solving problems where the input is text, characters, or sequences of symbols.

A string problem may ask you to:

- compare strings
- search for a pattern
- count character frequencies
- find substrings
- check subsequences
- build a lexicographically smallest string
- compress a string
- remove characters
- parse numbers
- match prefixes and suffixes

Many string problems are not DP.

Example:

Valid Anagram

does not need DP.

It needs frequency counting.

Example:

Find All Anagrams in a String

does not need DP.

It needs sliding window + frequency.

Example:

Implement strStr

can be solved by KMP.

2. String Algorithms vs String DP

Area	String Algorithms	String DP
Main idea	scan, count, match, construct	optimize over states
Common state	frequency, pointer, stack, prefix table	$dp[i]$, $dp[i][j]$, $dp[l][r]$
Example	Group Anagrams	Edit Distance
Example	KMP	LCS
Example	Remove Duplicate Letters	Longest Palindromic Subsequence
Typical complexity	$O(n)$, $O(n \log n)$, $O(n + m)$	$O(n^2)$, $O(mn)$, $O(n^3)$

Use String Algorithms when the problem is mostly about pattern processing.

Use String DP when the problem asks for optimal subsequence/edit/palindrome decisions.

3. Major String Algorithm Families

Family	Core Idea	Example
Frequency counting	Count characters	Valid Anagram
Hash map grouping	Use signature key	Group Anagrams
Sliding window	Maintain substring window	Find All Anagrams
Two pointers	Compare ends or simulate edits	Valid Palindrome
Subsequence scan	Advance pointer on match	Is Subsequence
Prefix/suffix	Compare string boundaries	Longest Common Prefix
KMP	Prefix table for pattern search	strStr
Z algorithm	Match prefix from each index	Pattern matching
Stack strings	Remove/build while scanning	Remove Duplicate Letters
Lexicographic greedy	Build smallest/largest valid result	Smallest Subsequence

Family	Core Idea	Example
Simulation	Follow rules directly	Count and Say
Arithmetic strings	Digit-by-digit math	Add Strings

4. When to Use String Algorithms

Use string algorithms when the problem mentions:

4.1 Anagram / Rearrangement

Use frequency arrays or sorted keys.

Examples:

- Valid Anagram
- Group Anagrams
- Find All Anagrams in a String

4.2 Substring Search

Use:

- built-in find for simple use
- KMP for interview-level efficient matching
- rolling hash for multiple/powerful substring comparisons

4.3 Prefix or Suffix

Use:

- direct scan
- prefix function
- Z algorithm
- trie for many strings

4.4 Lexicographic Smallest / Largest

Use greedy + stack.

Examples:

- Remove Duplicate Letters
- Smallest Subsequence of Distinct Characters

4.5 Adjacent Removal

Use stack.

Examples:

- Remove All Adjacent Duplicates
- Remove K Digits

4.6 Manual Arithmetic

Use digit-by-digit simulation.

Examples:

- Add Strings
- Multiply Strings

5. When Not to Use String Algorithms Alone

Do not force simple string algorithms when:

5.1 It Needs Edit Operations

Use String DP.

Example:

Edit Distance

5.2 It Needs Longest Common Subsequence

Use String DP.

5.3 It Needs Word Prefix Dictionary Search

Use Trie.

Example:

Word Search II

Search Suggestions System

5.4 It Needs Many Hash-Based Substring Comparisons

Use Rolling Hash.

5.5 It Needs Parsing Nested Expressions

Use stack / recursion / parsing patterns.

Parsing has its own chapter later.

6. Mental Models

6.1 Character Frequency

If order does not matter, count characters.

anagram

frequency equality

rearrangement

6.2 Order Preserving

If order matters but not contiguity, think subsequence.

Is Subsequence

6.3 Contiguous Segment

If the problem asks for substring, think sliding window, hashing, or KMP.

6.4 Prefix Reuse

If pattern matching repeatedly compares the same prefix, use KMP or Z algorithm.

6.5 Greedy Stack

If you need lexicographically smallest result after removals, use stack and last occurrence / remaining count.

7. Pattern Recognition Checklist

Ask:

26. Does order matter?

27. Is the target contiguous or non-contiguous?

28. Is this an anagram/frequency problem?

- 29. Can a frequency array of size 26 or 128 solve it?
 - 30. Is there a sliding window over substrings?
 - 31. Are we searching for pattern inside text?
 - 32. Do we repeatedly compare prefixes?
 - 33. Is lexicographic order important?
 - 34. Can stack help remove or build characters?
 - 35. Are we doing manual digit/string arithmetic?
 - 36. Is this actually DP instead?
 - 37. Is this actually Trie/Rolling Hash instead?
-

8. C++ Tools Needed

8.1 Character Count Array

```
vector<int> freq(26, 0);  
  
for (char c : s) {  
    freq[c - 'a']++;  
}
```

For ASCII:

```
vector<int> freq(128, 0);
```

For full byte range:

```
vector<int> freq(256, 0);
```

8.2 String Sort

```
sort(s.begin(), s.end());
```

8.3 Substring

```
s.substr(start, length);
```

Be careful: substring creation costs $O(\text{length})$.

8.4 String Builder

```
string result;  
result.push_back(c);  
result.pop_back();
```

8.5 Reverse

```
reverse(s.begin(), s.end());
```

8.6 Find

```
s.find(pattern)
```

Good for practical use, but interviewers may ask for KMP.

9. Core Templates

Template 1 — Frequency Count

```
vector<int> count(26, 0);

for (char c : s) {
    count[c - 'a']++;
}
```

Used in:

- Valid Anagram
 - Custom Sort String
 - character replacement variants
-

Template 2 — Sliding Window Frequency

```
vector<int> need(26, 0);
vector<int> window(26, 0);

for (char c : pattern) {
    need[c - 'a']++;
}

for (int right = 0; right < s.size(); right++) {
    window[s[right] - 'a']++;

    if (right >= pattern.size()) {
        window[s[right - pattern.size()] - 'a']--;
    }

    if (window == need) {
        // found match
    }
}
```

Used in:

- Find All Anagrams in a String
 - Permutation in String
-

Template 3 — Subsequence Check

```
int i = 0;

for (char c : t) {
    if (i < s.size() && s[i] == c) {
        i++;
    }
}

return i == s.size();
```

Template 4 — Palindrome Two Pointers

```

int left = 0;
int right = s.size() - 1;

while (left < right) {
    if (s[left] != s[right]) {
        return false;
    }

    left++;
    right--;
}

return true;

```

Template 5 — Stack String

```

string st;

for (char c : s) {
    if (!st.empty() && st.back() == c) {
        st.pop_back();
    } else {
        st.push_back(c);
    }
}

```

Template 6 — KMP Search

```

vector<int> lps = buildLPS(pattern);

int i = 0;
int j = 0;

while (i < text.size()) {
    if (text[i] == pattern[j]) {
        i++;
        j++;

        if (j == pattern.size()) {
            return i - j;
        }
    } else {
        if (j > 0) {
            j = lps[j - 1];
        } else {
            i++;
        }
    }
}

```

```
}  
  
return -1;
```

10. Full Problem Walkthrough 1 — LeetCode 242: Valid Anagram

10.1 Problem Summary

Given two strings *s* and *t*, return true if *t* is an anagram of *s*.
An anagram uses the same characters with same frequencies.

10.2 Pattern

Order does not matter.
Frequency matters.
Use count array.

10.3 C++ Code

```
class Solution {  
public:  
    bool isAnagram(string s, string t) {  
        if (s.size() != t.size()) {  
            return false;  
        }  
  
        vector<int> count(26, 0);  
  
        for (char c : s) {  
            count[c - 'a']++;  
        }  
  
        for (char c : t) {  
            count[c - 'a']--;  
        }  
  
        for (int x : count) {  
            if (x != 0) {  
                return false;  
            }  
        }  
  
        return true;  
    }  
};
```

10.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

because alphabet size is fixed.

11. Full Problem Walkthrough 2 — LeetCode 49: Group Anagrams

11.1 Problem Summary

Group words that are anagrams.

Example:

```
["eat", "tea", "tan", "ate", "nat", "bat"]
```

Output groups:

```
["eat", "tea", "ate"]  
["tan", "nat"]  
["bat"]
```

11.2 Approach 1 — Sorted Key

Anagrams become equal after sorting.

```
eat -> aet  
tea -> aet  
ate -> aet
```

11.3 C++ Code

```
class Solution {  
public:  
    vector<vector<string>> groupAnagrams(vector<string>& strs) {  
        unordered_map<string, vector<string>> groups;  
  
        for (string word : strs) {  
            string key = word;  
            sort(key.begin(), key.end());  
  
            groups[key].push_back(word);  
        }  
  
        vector<vector<string>> result;  
  
        for (auto& [key, group] : groups) {  
            result.push_back(group);  
        }  
  
        return result;  
    }  
};
```

11.4 Complexity

Let:

n = number of words
k = max word length

Time:

$O(n * k \log k)$

Space:

$O(nk)$

11.5 Approach 2 — Frequency Key

For lowercase letters, frequency key can be faster.

```
string buildKey(string word) {  
    vector<int> count(26, 0);  
  
    for (char c : word) {  
        count[c - 'a']++;  
    }  
  
    string key;  
  
    for (int x : count) {  
        key += '#';  
        key += to_string(x);  
    }  
  
    return key;  
}
```

This avoids sorting each word.

12. Full Problem Walkthrough 3 — LeetCode 438: Find All Anagrams in a String

12.1 Problem Summary

Given strings s and p, find all start indices of p's anagrams in s.

12.2 Pattern

Need all substrings of length p.size() with same frequency as p.

Use fixed-size sliding window.

12.3 C++ Code

```
class Solution {  
public:  
    vector<int> findAnagrams(string s, string p) {
```

```

        vector<int> result;

        int n = s.size();
        int m = p.size();

        if (m > n) {
            return result;
        }

        vector<int> need(26, 0);
        vector<int> window(26, 0);

        for (char c : p) {
            need[c - 'a']++;
        }

        for (int right = 0; right < n; right++) {
            window[s[right] - 'a']++;

            if (right >= m) {
                window[s[right - m] - 'a']--;
            }

            if (window == need) {
                result.push_back(right - m + 1);
            }
        }

        return result;
    }
};

```

12.4 Complexity

Time:

$O(26 * n)$

which is $O(n)$.

Space:

$O(1)$

13. Full Problem Walkthrough 4 — LeetCode 14: Longest Common Prefix

13.1 Problem Summary

Given array of strings, return longest common prefix.

13.2 Approach

Use first string as reference.
Compare each character across all strings.

13.3 C++ Code

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        if (strs.empty()) {
            return "";
        }

        string prefix = strs[0];

        for (int i = 1; i < strs.size(); i++) {
            while (strs[i].find(prefix) != 0) {
                prefix.pop_back();

                if (prefix.empty()) {
                    return "";
                }
            }
        }

        return prefix;
    }
};
```

13.4 Alternative Character-by-Character

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        if (strs.empty()) {
            return "";
        }

        for (int c = 0; c < strs[0].size(); c++) {
            char expected = strs[0][c];

            for (int i = 1; i < strs.size(); i++) {
                if (c == strs[i].size() || strs[i][c] != expected) {
                    return strs[0].substr(0, c);
                }
            }
        }

        return strs[0];
    }
};
```

14. Full Problem Walkthrough 5 — LeetCode 392: Is Subsequence

14.1 Problem Summary

Return true if s is a subsequence of t.

Example:

```
s = "abc"
t = "ahbgdc"
```

Answer:

```
true
```

14.2 Pattern

Use one pointer for s.

Scan t.

14.3 C++ Code

```
class Solution {
public:
    bool isSubsequence(string s, string t) {
        int i = 0;

        for (char c : t) {
            if (i < s.size() && s[i] == c) {
                i++;
            }
        }

        return i == s.size();
    }
};
```

14.4 Complexity

Time:

```
O(t.size())
```

Space:

```
O(1)
```

15. Full Problem Walkthrough 6 — LeetCode 844: Backspace String Compare

15.1 Problem Summary

Given two strings with # representing backspace, return whether final strings are equal.

15.2 Approach 1 — Build Final String

```
class Solution {
public:
    string build(string s) {
        string result;

        for (char c : s) {
            if (c == '#') {
                if (!result.empty()) {
                    result.pop_back();
                }
            } else {
                result.push_back(c);
            }
        }

        return result;
    }

    bool backspaceCompare(string s, string t) {
        return build(s) == build(t);
    }
};
```

15.3 Complexity

Time:

$O(n + m)$

Space:

$O(n + m)$

15.4 Approach 2 — Two Pointers From End

This avoids building strings.

```
class Solution {
public:
    int getNextValidIndex(string& s, int index) {
        int skip = 0;

        while (index >= 0) {
            if (s[index] == '#') {
                skip++;
                index--;
            } else if (skip > 0) {
                skip--;
                index--;
            } else {
                break;
            }
        }

        return index;
    }
};
```

```

        }
    }

    return index;
}

bool backspaceCompare(string s, string t) {
    int i = s.size() - 1;
    int j = t.size() - 1;

    while (i >= 0 || j >= 0) {
        i = getNextValidIndex(s, i);
        j = getNextValidIndex(t, j);

        if (i < 0 || j < 0) {
            return i == j;
        }

        if (s[i] != t[j]) {
            return false;
        }

        i--;
        j--;
    }

    return true;
}
};

```

16. Full Problem Walkthrough 7 — LeetCode 125: Valid Palindrome

16.1 Problem Summary

Check if string is palindrome after ignoring non-alphanumeric characters and case.

16.2 Pattern

Two pointers from both ends.

Skip invalid characters.

16.3 C++ Code

```

class Solution {
public:
    bool isPalindrome(string s) {
        int left = 0;
        int right = s.size() - 1;

        while (left < right) {

```

```

        while (left < right && !isalnum(s[left])) {
            left++;
        }

        while (left < right && !isalnum(s[right])) {
            right--;
        }

        if (tolower(s[left]) != tolower(s[right])) {
            return false;
        }

        left++;
        right--;
    }

    return true;
}
};

```

17. Full Problem Walkthrough 8 — LeetCode 680: Valid Palindrome II

17.1 Problem Summary

Return true if string can become palindrome after deleting at most one character.

17.2 Pattern

Two pointers.

At first mismatch, try skipping either left or right.

17.3 C++ Code

```

class Solution {
public:
    bool check(string& s, int left, int right) {
        while (left < right) {
            if (s[left] != s[right]) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }

    bool validPalindrome(string s) {
        int left = 0;

```

```
int right = s.size() - 1;

while (left < right) {
    if (s[left] != s[right]) {
        return check(s, left + 1, right) ||
               check(s, left, right - 1);
    }

    left++;
    right--;
}

return true;
};
```

17.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

18. KMP Introduction

KMP stands for:

Knuth-Morris-Pratt

It is used for pattern matching.

Problem:

Find pattern inside text.

Naive approach:

Try matching pattern from every text index.

Worst-case:

$O(nm)$

KMP improves to:

$O(n + m)$

by avoiding repeated comparisons.

19. Why KMP Works

Suppose pattern is:

ababaca

While matching text, if mismatch happens after matching:

ababa

we do not restart from scratch.

Because part of the matched prefix is also a suffix.

KMP uses this information to shift pattern efficiently.

This information is stored in the LPS array.

20. What Is LPS?

LPS means:

Longest Proper Prefix which is also Suffix

For every index i in pattern:

$\text{lps}[i]$ = length of longest proper prefix of $\text{pattern}[0..i]$
that is also a suffix of $\text{pattern}[0..i]$

Proper prefix means prefix not equal to the whole string.

Example:

`pattern = "abab"`

Prefixes:

a
ab
aba

Suffixes:

b
ab
bab

Longest prefix-suffix:

ab

So LPS for whole string is 2.

21. Full Problem Walkthrough 9 — Build KMP LPS Array

21.1 C++ Code

```
vector<int> buildLPS(string pattern) {  
    int n = pattern.size();  
  
    vector<int> lps(n, 0);  
  
    int len = 0;  
    int i = 1;  
  
    while (i < n) {
```

```

        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len > 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    return lps;
}

```

21.2 How to Understand It

len means:

current length of matching prefix-suffix

If characters match, extend it.

If mismatch, fall back to shorter prefix-suffix:

```
len = lps[len - 1];
```

This is the heart of KMP.

22. Full Problem Walkthrough 10 — LeetCode 28: Implement strStr

22.1 Problem Summary

Return index of first occurrence of needle in haystack.

22.2 KMP Code

```

class Solution {
public:
    vector<int> buildLPS(string pattern) {
        int n = pattern.size();

        vector<int> lps(n, 0);

        int len = 0;

        for (int i = 1; i < n; ) {
            if (pattern[i] == pattern[len]) {
                len++;
                lps[i] = len;
                i++;
            }
        }
    }
}

```

```

        } else {
            if (len > 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    return lps;
}

int strStr(string haystack, string needle) {
    if (needle.empty()) {
        return 0;
    }

    vector<int> lps = buildLPS(needle);

    int i = 0;
    int j = 0;

    while (i < haystack.size()) {
        if (haystack[i] == needle[j]) {
            i++;
            j++;

            if (j == needle.size()) {
                return i - j;
            }
        } else {
            if (j > 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }

    return -1;
}
};

```

22.3 Complexity

Time:

$O(n + m)$

Space:

$O(m)$

23. Full Problem Walkthrough 11 — LeetCode 459: Repeated Substring Pattern

23.1 Problem Summary

Return true if string can be constructed by repeating one of its substrings.

Examples:

```
abab -> true
aba -> false
abcbabcabc -> true
```

23.2 KMP Insight

If a string has repeated pattern, its LPS at the last index tells us the repeated structure.

Let:

```
n = s.size()
len = lps[n - 1]
```

If:

```
len > 0
n % (n - len) == 0
```

then repeated substring exists.

23.3 C++ Code

```
class Solution {
public:
    vector<int> buildLPS(string s) {
        int n = s.size();

        vector<int> lps(n, 0);

        int len = 0;

        for (int i = 1; i < n; ) {
            if (s[i] == s[len]) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len > 0) {
                    len = lps[len - 1];
                } else {
                    i++;
                }
            }
        }
    }
};
```

```

        }
    }
}

return lps;
}

bool repeatedSubstringPattern(string s) {
    int n = s.size();

    vector<int> lps = buildLPS(s);

    int len = lps[n - 1];

    return len > 0 && n % (n - len) == 0;
}
};

```

24. Full Problem Walkthrough 12 — LeetCode 1392: Longest Happy Prefix

24.1 Problem Summary

A happy prefix is a non-empty prefix that is also a suffix, but not the whole string. Return the longest happy prefix.

24.2 KMP Insight

The last value of LPS array gives exactly this.

24.3 C++ Code

```

class Solution {
public:
    vector<int> buildLPS(string s) {
        int n = s.size();

        vector<int> lps(n, 0);

        int len = 0;

        for (int i = 1; i < n; ) {
            if (s[i] == s[len]) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len > 0) {
                    len = lps[len - 1];
                } else {
                    i++;
                }
            }
        }
    }
};

```

```

        }
    }
}

return lps;
}

string longestPrefix(string s) {
    vector<int> lps = buildLPS(s);

    int len = lps.back();

    return s.substr(0, len);
}
};

```

25. Z Algorithm Preview

25.1 What Is Z Array?

For string s:

$z[i]$ = length of longest substring starting at i
that matches prefix of s

Example:

$s = \text{"aaaa"}$

Z array:

$z[0] = 0$ by convention
 $z[1] = 3$
 $z[2] = 2$
 $z[3] = 1$

25.2 Why Use Z Algorithm?

It can solve pattern matching by building:

pattern + '#' + text

Then wherever $z[i]$ equals pattern length, pattern occurs.

25.3 Z Algorithm Code

```

vector<int> buildZ(string s) {
    int n = s.size();

    vector<int> z(n, 0);

    int left = 0;
    int right = 0;

    for (int i = 1; i < n; i++) {
        if (i <= right) {

```

```

        z[i] = min(right - i + 1, z[i - left]);
    }

    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
        z[i]++;
    }

    if (i + z[i] - 1 > right) {
        left = i;
        right = i + z[i] - 1;
    }
}

return z;
}

```

25.4 KMP vs Z Algorithm

KMP	Z Algorithm
uses prefix function / LPS	uses prefix match lengths
great for pattern search	great for prefix matching
common in interviews	more common in competitive programming
$O(n + m)$	$O(n + m)$

26. Full Problem Walkthrough 13 — LeetCode 796: Rotate String

26.1 Problem Summary

Return true if s can become goal after some rotations.

Example:

```

s = "abcde"
goal = "cdeab"

```

Answer:

```
true
```

26.2 Key Trick

If goal is a rotation of s, then goal appears inside:

```
s + s
```

26.3 C++ Code

```

class Solution {
public:
    bool rotateString(string s, string goal) {

```

```

        if (s.size() != goal.size()) {
            return false;
        }

        string doubled = s + s;

        return doubled.find(goal) != string::npos;
    }
};

```

26.4 With KMP

If interviewer asks not to use built-in find, use KMP search on `s + s`.

27. Full Problem Walkthrough 14 — LeetCode 443: String Compression

27.1 Problem Summary

Compress character array in-place.

Example:

```
["a", "a", "b", "b", "c", "c", "c"]
```

becomes:

```
["a", "2", "b", "2", "c", "3"]
```

Return new length.

27.2 Pattern

Two pointers:

- read pointer scans groups
- write pointer writes compressed result

27.3 C++ Code

```

class Solution {
public:
    int compress(vector<char>& chars) {
        int n = chars.size();

        int write = 0;
        int read = 0;

        while (read < n) {
            char current = chars[read];
            int count = 0;

            while (read < n && chars[read] == current) {
                read++;
                count++;
            }

            chars[write] = current;
            write++;

            if (count > 1) {
                string countStr = to_string(count);
                for (char c : countStr) {
                    chars[write] = c;
                    write++;
                }
            }
        }

        return write;
    }
};

```

```

        chars[write++] = current;

        if (count > 1) {
            string countStr = to_string(count);

            for (char c : countStr) {
                chars[write++] = c;
            }
        }

        return write;
    }
};

```

28. Full Problem Walkthrough 15 — LeetCode 1047: Remove All Adjacent Duplicates in String

28.1 Problem Summary

Repeatedly remove adjacent equal characters.

Example:

abbaca

Process:

abbaca -> aaca -> ca

Answer:

ca

28.2 Pattern

Use string as stack.

28.3 C++ Code

```

class Solution {
public:
    string removeDuplicates(string s) {
        string st;

        for (char c : s) {
            if (!st.empty() && st.back() == c) {
                st.pop_back();
            } else {
                st.push_back(c);
            }
        }
    }
};

```

```
        return st;
    }
};
```

29. Full Problem Walkthrough 16 — LeetCode 316: Remove Duplicate Letters

29.1 Problem Summary

Remove duplicate letters so every letter appears once and result is lexicographically smallest.

Example:

```
s = "cbacdcbc"
```

Answer:

```
"acdb"
```

29.2 Why This Is Hard

Need:

- each character once
- smallest lexicographic order
- cannot remove a character if it will not appear later again

29.3 Greedy Stack Idea

Maintain stack.

For current character c:

While stack top is larger than c and stack top appears later again, pop it.

Then push c if not already used.

29.4 C++ Code

```
class Solution {
public:
    string removeDuplicateLetters(string s) {
        vector<int> last(26, 0);

        for (int i = 0; i < s.size(); i++) {
            last[s[i] - 'a'] = i;
        }

        vector<bool> used(26, false);
        string st;

        for (int i = 0; i < s.size(); i++) {
            char c = s[i];
            int idx = c - 'a';
```

```

        if (used[idx]) {
            continue;
        }

        while (!st.empty() &&
               st.back() > c &&
               last[st.back() - 'a'] > i) {
            used[st.back() - 'a'] = false;
            st.pop_back();
        }

        st.push_back(c);
        used[idx] = true;
    }

    return st;
}
};

```

29.5 Key Explanation

We can pop a larger character only if it appears again later.

Otherwise, popping it would make it impossible to include all characters.

30. Full Problem Walkthrough 17 — LeetCode 1249: Minimum Remove to Make Valid Parentheses

30.1 Problem Summary

Remove minimum number of parentheses so the string becomes valid.

30.2 Pattern

Use stack to track unmatched opening parentheses.

Use set/boolean array to mark invalid positions.

30.3 C++ Code

```

class Solution {
public:
    string minRemoveToMakeValid(string s) {
        stack<int> st;
        vector<bool> remove(s.size(), false);

        for (int i = 0; i < s.size(); i++) {
            if (s[i] == '(') {
                st.push(i);
            } else if (s[i] == ')') {
                if (!st.empty()) {

```

```

        st.pop();
    } else {
        remove[i] = true;
    }
}

while (!st.empty()) {
    remove[st.top()] = true;
    st.pop();
}

string result;

for (int i = 0; i < s.size(); i++) {
    if (!remove[i]) {
        result.push_back(s[i]);
    }
}

return result;
}
};

```

31. Full Problem Walkthrough 18 — LeetCode 791: Custom Sort String

31.1 Problem Summary

Given order and s, sort s according to custom character order.

31.2 Pattern

Frequency count.

Output characters in order first, then remaining.

31.3 C++ Code

```

class Solution {
public:
    string customSortString(string order, string s) {
        vector<int> count(26, 0);

        for (char c : s) {
            count[c - 'a']++;
        }

        string result;

        for (char c : order) {
            while (count[c - 'a'] > 0) {

```

```

        result.push_back(c);
        count[c - 'a']--;
    }
}

for (int i = 0; i < 26; i++) {
    while (count[i] > 0) {
        result.push_back('a' + i);
        count[i]--;
    }
}

return result;
}
};

```

32. Full Problem Walkthrough 19 — LeetCode 415: Add Strings

32.1 Problem Summary

Add two non-negative integers represented as strings.
Do not convert to integer.

32.2 Pattern

Simulate addition from right to left.

32.3 C++ Code

```

class Solution {
public:
    string addStrings(string num1, string num2) {
        int i = num1.size() - 1;
        int j = num2.size() - 1;
        int carry = 0;

        string result;

        while (i >= 0 || j >= 0 || carry > 0) {
            int x = 0;
            int y = 0;

            if (i >= 0) {
                x = num1[i] - '0';
                i--;
            }

            if (j >= 0) {
                y = num2[j] - '0';
                j--;
            }

```

```

        }

        int sum = x + y + carry;

        result.push_back('0' + sum % 10);
        carry = sum / 10;
    }

    reverse(result.begin(), result.end());

    return result;
}
};

```

33. Full Problem Walkthrough 20 — LeetCode 43: Multiply Strings

33.1 Problem Summary

Multiply two non-negative integers represented as strings.

Do not convert to integer.

33.2 Pattern

Grade-school multiplication.

If:

```
num1[i] * num2[j]
```

then result positions are:

```

i + j
i + j + 1

```

33.3 C++ Code

```

class Solution {
public:
    string multiply(string num1, string num2) {
        if (num1 == "0" || num2 == "0") {
            return "0";
        }

        int m = num1.size();
        int n = num2.size();

        vector<int> result(m + n, 0);

        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                int product = (num1[i] - '0') * (num2[j] - '0');

```

```

        int p1 = i + j;
        int p2 = i + j + 1;

        int sum = product + result[p2];

        result[p2] = sum % 10;
        result[p1] += sum / 10;
    }
}

string answer;

int i = 0;

while (i < result.size() && result[i] == 0) {
    i++;
}

while (i < result.size()) {
    answer.push_back('0' + result[i]);
    i++;
}

return answer.empty() ? "0" : answer;
}
};

```

34. Full Problem Walkthrough 21 — LeetCode 165: Compare Version Numbers

34.1 Problem Summary

Compare two version strings.

Example:

"1.01" and "1.001"

They are equal because numeric parts are both 1 and 1.

34.2 Pattern

Parse integer parts separated by dots.

34.3 C++ Code

```

class Solution {
public:
    int compareVersion(string version1, string version2) {
        int i = 0;
        int j = 0;
    }
};

```

```

int n = version1.size();
int m = version2.size();

while (i < n || j < m) {
    long long num1 = 0;
    long long num2 = 0;

    while (i < n && version1[i] != '.') {
        num1 = num1 * 10 + (version1[i] - '0');
        i++;
    }

    while (j < m && version2[j] != '.') {
        num2 = num2 * 10 + (version2[j] - '0');
        j++;
    }

    if (num1 < num2) {
        return -1;
    }

    if (num1 > num2) {
        return 1;
    }

    i++;
    j++;
}

return 0;
};

```

35. Full Problem Walkthrough 22 — LeetCode 38: Count and Say

35.1 Problem Summary

Generate nth term of count-and-say sequence.

Example:

```

1
11
21
1211
111221

```

35.2 Pattern

Simulation.

Read groups of identical characters.

35.3 C++ Code

```
class Solution {
public:
    string countAndSay(int n) {
        string current = "1";

        for (int step = 2; step <= n; step++) {
            string next;

            int i = 0;

            while (i < current.size()) {
                char c = current[i];
                int count = 0;

                while (i < current.size() && current[i] == c) {
                    count++;
                    i++;
                }

                next += to_string(count);
                next.push_back(c);
            }

            current = next;
        }

        return current;
    }
};
```

36. Full Problem Walkthrough 23 — Roman to Integer

36.1 Problem Summary

Convert Roman numeral to integer.

Special subtraction cases:

```
IV = 4
IX = 9
XL = 40
XC = 90
CD = 400
CM = 900
```

36.2 Pattern

If current value is less than next value, subtract it.

Otherwise add it.

36.3 C++ Code

```
class Solution {
public:
    int romanToInt(string s) {
        unordered_map<char, int> value;

        value['I'] = 1;
        value['V'] = 5;
        value['X'] = 10;
        value['L'] = 50;
        value['C'] = 100;
        value['D'] = 500;
        value['M'] = 1000;

        int answer = 0;

        for (int i = 0; i < s.size(); i++) {
            if (i + 1 < s.size() && value[s[i]] < value[s[i + 1]]) {
                answer -= value[s[i]];
            } else {
                answer += value[s[i]];
            }
        }

        return answer;
    }
};
```

37. Common String Edge Cases

37.1 Empty String

Important for:

- longest common prefix
- strStr
- palindrome
- compression

37.2 One Character

Important for:

- palindrome
- stack removal
- repeated substring pattern

37.3 Case Sensitivity

Ask:

Are uppercase and lowercase different?

37.4 Non-Alphanumeric Characters

Important for:

- Valid Palindrome
- parsing problems

37.5 Unicode

Most interview problems assume ASCII or lowercase English letters.

If not, frequency arrays may not work.

37.6 Large Strings

Avoid repeated substring creation in loops if n is large.

38. Common String Algorithm Mistakes

Mistake 1 — Using Sorting When Frequency Is Enough

Sorting is $O(k \log k)$.

Frequency can be $O(k)$.

Mistake 2 — Confusing Substring and Subsequence

Substring is contiguous.

Subsequence can skip characters.

Mistake 3 — Repeated substr in Loop

This can turn $O(n^2)$ into $O(n^3)$.

Mistake 4 — KMP LPS Off-by-One

When mismatch occurs:

```
len = lps[len - 1];
```

not:

```
len--;
```

Mistake 5 — Incorrect Lexicographic Stack Pop

For Remove Duplicate Letters, only pop if the popped character appears later again.

Mistake 6 — Forgetting Carry

String arithmetic needs carry handling.

39. Interview Explanation Templates

39.1 Frequency String Problem

Since order does not matter and only character counts matter, I count frequencies using a fixed-size array.

39.2 Sliding Window Anagram

I maintain a window of length equal to the pattern and compare its frequency with the pattern frequency.

39.3 Subsequence

I scan the larger string and advance a pointer in the smaller string whenever characters match.

39.4 KMP

I build an LPS array for the pattern. During matching, when a mismatch occurs, I use the LPS value to avoid rechecking characters that are already known to match.

39.5 Greedy Stack String

I maintain a stack-like result. If the current character is smaller than the stack top and the stack top appears later, I pop it to improve lexicographic order.

40. Complexity Patterns

Problem Type	Common Time	Common Space
Frequency count	$O(n)$	$O(1)$
Group anagrams sorted key	$O(nk \log k)$	$O(nk)$
Group anagrams frequency key	$O(nk)$	$O(nk)$
Sliding window anagram	$O(n)$	$O(1)$
Subsequence check	$O(n)$	$O(1)$
Palindrome two pointers	$O(n)$	$O(1)$
KMP search	$O(n + m)$	$O(m)$
Z algorithm	$O(n + m)$	$O(n + m)$
Stack string	$O(n)$	$O(n)$
String addition	$O(n + m)$	$O(n + m)$
String multiplication	$O(nm)$	$O(n + m)$

41. Expanded Practice Set

41.1 Must-Do Easy / Medium

- 38. Valid Anagram
- 39. Group Anagrams
- 40. Find All Anagrams in a String
- 41. Longest Common Prefix
- 42. Is Subsequence
- 43. Backspace String Compare
- 44. Valid Palindrome
- 45. Valid Palindrome II
- 46. Implement strStr
- 47. Repeated Substring Pattern
- 48. Rotate String

- 49. String Compression
- 50. Remove All Adjacent Duplicates
- 51. Custom Sort String
- 52. Add Strings
- 53. Compare Version Numbers

41.2 Strong Interview Practice

- 54. Remove Duplicate Letters
- 55. Smallest Subsequence of Distinct Characters
- 56. Minimum Remove to Make Valid Parentheses
- 57. Longest Happy Prefix
- 58. KMP Pattern Matching
- 59. Z Algorithm Pattern Matching
- 60. Multiply Strings
- 61. Count and Say
- 62. Roman to Integer
- 63. Integer to Roman

41.3 Hard / Stretch

- 64. Shortest Palindrome using KMP
 - 65. Repeated String Match
 - 66. Find the Index of First Occurrence using KMP
 - 67. Longest Duplicate Substring, rolling hash later
 - 68. Distinct Echo Substrings, rolling hash later
 - 69. Minimum Window Substring, sliding window review
 - 70. Basic Calculator, parsing later
 - 71. Decode String, stack/parsing later
-

42. Suggested Study Order

Step 1 — Frequency Basics

Solve:

- 72. Valid Anagram
- 73. Group Anagrams
- 74. Custom Sort String

Goal:

Master character counts and signature keys.

Step 2 — Sliding Window Strings

Solve:

- 75. Find All Anagrams
- 76. Permutation in String
- 77. Minimum Window Substring review

Goal:

Use frequency inside windows.

Step 3 — Two Pointers

Solve:

- 78. Valid Palindrome
- 79. Valid Palindrome II
- 80. Backspace String Compare

Goal:

Master boundary and skip logic.

Step 4 — KMP

Solve:

- 81. Implement strStr
- 82. Repeated Substring Pattern
- 83. Longest Happy Prefix

Goal:

Understand LPS deeply.

Step 5 — Stack / Greedy Strings

Solve:

- 84. Remove Adjacent Duplicates
- 85. Remove Duplicate Letters
- 86. Minimum Remove to Make Valid Parentheses

Goal:

Learn stack-based construction.

Step 6 — String Arithmetic / Simulation

Solve:

- 87. Add Strings
- 88. Multiply Strings
- 89. Compare Version Numbers
- 90. Count and Say

Goal:

Improve careful scan and simulation skills.

43. Pattern Decision Table

Problem Clue	Best Technique
same characters different order	frequency / sorting
group by anagram	sorted key / frequency key
all anagrams in text	sliding window frequency
prefix common to all strings	vertical scan / sorting
pattern in text	KMP
prefix equals suffix	LPS / KMP
rotation	check in s + s
palindrome ignoring chars	two pointers
delete at most one char palindrome	two pointers + helper
adjacent duplicate removals	stack

Problem Clue	Best Technique
lexicographically smallest unique chars	greedy stack
valid parentheses removal	stack / marking
add numeric strings	right-to-left carry
multiply numeric strings	digit array simulation
compare dotted versions	parse parts

44. Mini Problem 1 — Permutation in String

Given s1 and s2, return true if s2 contains any permutation of s1.
Same as anagram window.

```
class Solution {
public:
    bool checkInclusion(string s1, string s2) {
        int m = s1.size();
        int n = s2.size();

        if (m > n) {
            return false;
        }

        vector<int> need(26, 0);
        vector<int> window(26, 0);

        for (char c : s1) {
            need[c - 'a']++;
        }

        for (int right = 0; right < n; right++) {
            window[s2[right] - 'a']++;

            if (right >= m) {
                window[s2[right - m] - 'a']--;
            }

            if (window == need) {
                return true;
            }
        }

        return false;
    }
};
```

```
    }  
};
```

45. Mini Problem 2 — Reverse Words in a String

```
class Solution {  
public:  
    string reverseWords(string s) {  
        vector<string> words;  
  
        int i = 0;  
        int n = s.size();  
  
        while (i < n) {  
            while (i < n && s[i] == ' ') {  
                i++;  
            }  
  
            if (i == n) {  
                break;  
            }  
  
            int start = i;  
  
            while (i < n && s[i] != ' ') {  
                i++;  
            }  
  
            words.push_back(s.substr(start, i - start));  
        }  
  
        reverse(words.begin(), words.end());  
  
        string result;  
  
        for (int j = 0; j < words.size(); j++) {  
            if (j > 0) {  
                result.push_back(' ');  
            }  
  
            result += words[j];  
        }  
  
        return result;  
    }  
};
```

```
}  
};
```

46. Mini Problem 3 — Shortest Palindrome Using KMP Preview

Problem:

Add characters in front of s to make it palindrome.

Key idea:

Find longest palindromic prefix.

Build:

```
s + "#" + reverse(s)
```

The last LPS value gives longest prefix of s that matches suffix of reverse(s).

```
class Solution {  
public:  
    vector<int> buildLPS(string p) {  
        vector<int> lps(p.size(), 0);  
  
        int len = 0;  
  
        for (int i = 1; i < p.size(); ) {  
            if (p[i] == p[len]) {  
                lps[i++] = ++len;  
            } else if (len > 0) {  
                len = lps[len - 1];  
            } else {  
                i++;  
            }  
        }  
  
        return lps;  
    }  
  
    string shortestPalindrome(string s) {  
        string rev = s;  
        reverse(rev.begin(), rev.end());  
  
        string combined = s + "#" + rev;  
  
        vector<int> lps = buildLPS(combined);  
  
        int palPrefixLen = lps.back();  
  
        string add = s.substr(palPrefixLen);  
        reverse(add.begin(), add.end());
```

```
        return add + s;
    }
};
```

47. Mini Problem 4 — Repeated String Match

Given strings a and b, return minimum times a must be repeated so b is substring.

```
class Solution {
public:
    int repeatedStringMatch(string a, string b) {
        string repeated;
        int count = 0;

        while (repeated.size() < b.size()) {
            repeated += a;
            count++;
        }

        if (repeated.find(b) != string::npos) {
            return count;
        }

        repeated += a;
        count++;

        if (repeated.find(b) != string::npos) {
            return count;
        }

        return -1;
    }
};
```

For strict algorithmic version, replace find with KMP.

48. Common Edge Case Checklist

Before submitting string code, test:

```
" "
"a"
"aa"
"ab"
"aba"
"abba"
"abc"
```

strings with spaces
strings with punctuation
uppercase and lowercase
pattern longer than text
pattern equal to text
all same characters
all unique characters
repeated prefix/suffix
numeric string with carry
numeric string "0"
version string with leading zeros

49. Debugging String Algorithms

If your string solution fails, check:

91. Are you treating substring and subsequence correctly?
92. Are character frequencies initialized/reset correctly?
93. Is your window size exactly correct?
94. Are you skipping non-alphanumeric characters correctly?
95. Are uppercase/lowercase handled correctly?
96. Are you creating substrings inside nested loops?
97. Is your KMP LPS table correct?
98. Are stack pops safe?
99. Is lexicographic popping allowed only when character appears later?
100. Are carry and leading zeros handled in arithmetic strings?

50. What to Master Before Chapter 45

Before moving to Palindrome Patterns, make sure you can:

- Solve Valid Anagram
- Solve Group Anagrams
- Solve Find All Anagrams
- Solve Is Subsequence
- Solve Valid Palindrome
- Solve Valid Palindrome II
- Build KMP LPS array
- Implement strStr using KMP
- Solve Repeated Substring Pattern
- Solve Longest Happy Prefix
- Use stack for adjacent duplicate removals
- Solve Remove Duplicate Letters
- Add numeric strings
- Multiply numeric strings
- Choose between frequency, sliding window, KMP, stack, and DP

51. Final Cheat Sheet

Frequency

```
vector<int> count(26, 0);

for (char c : s) {
    count[c - 'a']++;
}
```

Sliding Anagram Window

```
window[s[right] - 'a']++;

if (right >= m) {
    window[s[right - m] - 'a']--;
}
```

Subsequence

```
if (i < s.size() && s[i] == c) {
    i++;
}
```

Palindrome

```
while (left < right) {
    if (s[left] != s[right]) {
        return false;
    }

    left++;
    right--;
}
```

KMP Fallback

```
j = lps[j - 1];
```

Repeated Substring

```
len = lps[n - 1];

return len > 0 && n % (n - len) == 0;
```

Rotation

```
return s.size() == goal.size() &&
    (s + s).find(goal) != string::npos;
```

Stack String

```
if (!st.empty() && st.back() == c) {
    st.pop_back();
} else {
    st.push_back(c);
}
```

Remove Duplicate Letters

```
while (!st.empty() &&
      st.back() > c &&
      last[st.back() - 'a'] > i) {
    used[st.back() - 'a'] = false;
    st.pop_back();
}
```

52. Core Takeaway

String Algorithms are about choosing the right processing pattern.

Ask first:

Does order matter?
Is it substring or subsequence?
Is it frequency-based?
Is it prefix/suffix-based?
Is it lexicographic?
Is it pattern matching?

Then choose:

- frequency array for anagrams
- sliding window for substring frequency
- two pointers for palindrome/comparison
- KMP for pattern matching
- stack for removals and greedy string building
- simulation for arithmetic and parsing-like strings
- DP only when optimal edit/subsequence/range decisions are needed

String problems become much easier when you classify the pattern before coding.